ISE 572: Industry 4.0 Machine Learning

Final Project Proposal

Machine Learning for Shipyard Production Scheduling:
Neural Network-Based Block Sequencing and Processing Time Prediction

Stephen Eacuello Steven Blum

February 2026

Section 1: Problem Statement

The Industry 4.0 Problem

Modern shipyards such as Hyundai Heavy Industries (HHI) Ulsan must coordinate hundreds of steel blocks through a multi-stage production pipeline involving steel cutting, panel assembly, block assembly, outfitting, painting, and final erection at the dry dock. Each block is transported by self-propelled modular transporters (SPMTs) and lifted into position by goliath cranes, all while respecting structural precedence constraints, equipment availability, and ship delivery deadlines. This is a large-scale, real-time **process optimization** problem squarely within the Industry 4.0 domain of smart manufacturing.

Why This Problem Matters

Poor scheduling decisions cascade through the entire production line. An idle goliath crane costs an estimated $10,000–15,000 per day in lost throughput. Late block delivery to the dock delays ship launches, triggering contractual penalties that can reach millions of dollars per vessel. Meanwhile, equipment failures from deferred maintenance cause unplanned downtime that disrupts the entire yard. Current practice relies on experienced human planners using heuristic rules and spreadsheets, an approach that does not scale as yards handle 5–10 concurrent ship builds with 200–400 blocks each.

Beyond direct financial costs, inefficient scheduling increases energy consumption (unnecessary SPMT travel, idle crane operation), extends worker exposure to hazardous environments, and creates downstream bottlenecks in outfitting and painting that compound over weeks. The International Maritime Organization's push toward shorter build cycles for next-generation LNG carriers and container ships makes this optimization increasingly urgent.

Industry Domain

This project applies to the **heavy manufacturing and maritime logistics** sector, specifically shipbuilding production management. The problem generalizes to any large-scale job-shop scheduling environment with heterogeneous equipment, precedence constraints, and stochastic processing times—including aerospace assembly, modular construction, and semiconductor fabrication.

Section 2: Data Considerations

Required Data

The model requires four categories of data, all of which exist or are generated within our simulation framework:

Equipment telemetry and status data. Real-time position, operational status (idle, transporting, lifting, under maintenance), and degradation health indicators for each SPMT and goliath crane. In a production deployment, this data would come from IoT sensors, GPS trackers, and SCADA systems installed on yard equipment. Our simulation models 6 SPMTs and 9 goliath cranes with realistic status transitions and Bayesian health degradation.

Block production records. For each of the ∼50–200 blocks per ship: current production stage, processing start/end times, assigned facility, weight, dimensions, and structural precedence relationships (which blocks must be placed before others). In practice, this comes from the yard's Manufacturing Execution System (MES) and ERP databases. Our environment tracks these as structured entities with full stage progression.

Plate-level decomposition geometry. Each block is composed of 10–50 individual steel plates characterized by dimensions (length, width, thickness in mm), plate type (flat, curved, stiffened, bracket, bulkhead, shell), number of stiffeners, curvature radius, and material grade. This data originates from 3D CAD/CAM design software (e.g., AVEVA Marine, NAPA) and is exported via our partner Steven's decomposition scripts as structured JSON. These geometric features are critical inputs for predicting per-stage processing times—a curved plate with stiffeners takes significantly longer to cut and weld than a flat plate.

Ship schedule and constraint data. Delivery deadlines, dock assignments, and the mapping between blocks and ships. Sourced from project management and contract databases.

Data Challenges

Sim-to-real gap. Our primary challenge is that we train on simulated data with synthetic plate geometries while real yards have noisy, heterogeneous data sources. We address this through a calibration pipeline that fits regression coefficients from observed processing times, and domain randomization that varies processing parameters during training to improve robustness.

Plate data availability. Full plate-level decomposition data requires CAD model post-processing that many yards do not routinely perform for scheduling purposes. We handle

this with a synthetic plate generation fallback that estimates plate counts and geometry from block weight and type, producing physically plausible distributions validated against published shipbuilding references.

Sparse reward signal. Block completions occur infrequently (every ∼50–100 scheduling steps), making it difficult to attribute credit to individual decisions. This is a classic challenge in sequential decision-making that we address through reward shaping and dense intermediate signals (transport completion, stage advancement).

Section 3: Proposed ML Solution

Problem Formulation

This project addresses **three interconnected ML problems**, each mapping to techniques covered in the course:

1. Regression: Processing Time Prediction from Plate Geometry

Given a block's plate-level characteristics, we predict the processing time (in hours) at each production stage. This is a **multivariate linear regression** problem:

$$t_{\text{stage}} = \beta_0 + \beta_1 \cdot n_{\text{plates}} + \beta_2 \cdot n_{\text{curved}} + \beta_3 \cdot n_{\text{stiffened}} + \beta_4 \cdot A_{\text{total}} + \beta_5 \cdot L_{\text{weld}} \tag{1}$$

where the features are plate count, curved plate count, stiffened plate count, total plate area (m^2), and total weld length (m). Coefficients are fitted per stage (steel cutting, panel assembly, block assembly, etc.) using least-squares optimization with non-negativity constraints. This replaces the current practice of using historical averages or lognormal distributions, giving geometry-informed estimates. Steven's plate decomposition data from CAD models provides the ground-truth features for fitting these models. We validate with R^2, RMSE, and MAE metrics using cross-validation across blocks.

2. Artificial Neural Networks: Scheduling Policy and Value Estimation

The core scheduling agent uses a deep **actor-critic neural network** architecture:

- **Actor network (policy):** A multi-layer perceptron (MLP) with ReLU activations that takes a learned state representation and outputs probability distributions over discrete actions. The network has a shared 2-layer trunk (256 hidden units) that branches into multiple classification heads: action type selection (4-way: dispatch SPMT, dispatch crane, schedule maintenance, or hold), equipment selection (which SPMT or crane), and target selection (which block/request to service). Each head applies softmax activation to produce a categorical probability distribution—this is a direct generalization of **logistic regression** to the multi-class setting, where a single-output sigmoid (binary logistic regression) extends to a K-way softmax over action categories. Additionally, the agent includes a binary **logistic regression classifier** (single sigmoid output) that predicts whether a block is ready to advance to its next production stage, used to gate dispatching decisions. Action masking sets infeasible actions to zero probability before sampling.

- **Critic network (value function):** A separate MLP head that estimates the expected future reward from the current state, used to compute advantage estimates for stable policy gradient training. The critic outputs a single scalar value via a 2-layer network ($256 \rightarrow 128 \rightarrow 1$).

The policy is trained using Proximal Policy Optimization (PPO) with clipped surrogate objectives, and alternatively with Soft Actor-Critic (SAC) using separate Q-networks for value estimation. We also employ DAgger (Dataset Aggregation), an imitation learning technique where the neural network learns from an expert scheduling heuristic and iteratively improves by collecting on-policy data.

3. Graph Neural Networks: Relational State Encoding (CNN Analog)

The shipyard state has inherent graph structure—blocks connect to facilities, SPMTs connect to blocks they can transport, cranes connect to blocks they can lift, and blocks have precedence relationships with each other. A shipyard can also be represented as a 2D spatial grid (the physical yard map with equipment positions on a coordinate plane), making it amenable to standard **CNN** feature extraction over spatial tiles [3]. However, critical relationships—structural precedence between blocks, equipment-to-task eligibility, facility capacity constraints—are *non-spatial* and cannot be captured by a fixed-size convolutional kernel on a pixel grid. We therefore use a **heterogeneous graph neural network (GNN)** with Graph Attention (GAT) convolution layers, which *generalizes* the CNN to irregular, relational topologies while retaining the same core mechanism of learned local aggregation [2].

Concretely, GNNs perform *graph convolutions* that are the relational analog of spatial convolutions in CNNs: where a CNN slides a learned filter over a pixel grid to aggregate local spatial features, a GNN passes learned messages along graph edges to aggregate local neighborhood features. Our architecture uses:

- **Input projection layers:** Separate linear encodings per node type (blocks: 16 features including 4 plate-derived features, SPMTs: 10 features, cranes: 7 features, facilities: 3 features).
- **Message-passing layers:** Two layers of multi-head graph attention convolution (4 attention heads per layer) with residual connections and layer normalization, operating over 8 edge types (e.g., "block needs_transport SPMT", "block precedes block").
- **Global pooling:** Mean pooling per node type, concatenated into a fixed-size state embedding (512 dimensions) that feeds the actor-critic ANNs described above.

The 4 plate-derived features added by Steven's decomposition—normalized plate count, normalized plate area, percent curved plates, and percent stiffened plates—give the GNN richer block representations that capture manufacturing complexity beyond simple weight and size.

Baseline Comparisons

We compare the neural network approach against traditional operations research methods to establish the value added by the learned policy:

- **Earliest Due Date (EDD):** Rule-based heuristic (no ML)
- **Mixed-Integer Programming (MIP):** PuLP CBC solver with rolling-horizon optimization
- **Genetic Algorithm:** Evolutionary optimization with permutation chromosomes
- **Model Predictive Control:** CP-SAT constraint programming

Section 4: Expected Impact

Industry Benefits

A trained scheduling policy that outperforms expert heuristics would directly reduce block throughput time, equipment idle time, and delivery tardiness. Based on our preliminary experiments, the Expert EDD baseline achieves $\sim$32 blocks erected per episode (300 time steps) while the MIP optimizer achieves $\sim$25 blocks. An RL-trained policy that matches or exceeds Expert performance while also optimizing maintenance timing would provide:

- **Reduced idle costs:** Fewer idle crane-hours through better sequencing.
- **Faster throughput:** More blocks completed per unit time by optimizing SPMT routing and crane assignment jointly.
- **Predictive maintenance integration:** Scheduling maintenance during natural lulls rather than fixed intervals, reducing unplanned downtime.
- **Accurate time estimates:** Plate-geometry-driven processing time regression replacing rough historical averages, enabling tighter schedule planning.

The plate-level processing time model is independently valuable: giving planners per-stage time estimates based on actual block geometry rather than yard averages enables better capacity planning even without the RL policy.

Evaluation Metrics

We evaluate the system using metrics that are standard in both scheduling research and shipyard operations:

Additionally, we report wall-clock computation time per decision to verify the policy is fast enough for real-time deployment (target $< 10\,\text{ms}$ per action, vs. seconds for MIP solvers).

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[2] M. M. Bronstein, J. Bruna, T. Cohen, and P. Veličković, "Geometric deep learning: Grids, groups, graphs, and geodesics," *arXiv preprint arXiv:2104.13478*, 2021.

Metric	Definition	Target
Throughput	Blocks erected per time unit	Maximize ($\geq$ Expert)
Total tardiness	$\sum$(completion $-$ due date) for late blocks	Minimize
Equip. utilization	Fraction of time equipment is productively busy	$> 70\%$
Makespan	Time to complete all blocks for a ship	Minimize
Proc. time R^2	Regression accuracy of plate-based predictions	> 0.7 (real data)
RMSE	Root mean squared error of time predictions	< 2 hours/stage

Table 1: Evaluation metrics and targets.

[3] L. Wang, Z. Pan, and J. Wang, "A review of reinforcement learning based intelligent optimization for manufacturing scheduling," *Complex & Intelligent Systems*, vol. 7, no. 2, pp. 597–618, 2021.

[4] S. Ross, G. J. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proc. AISTATS*, pp. 627–635, 2011.

[5] J.-G. Shin, S. J. Lee, and Y. Kim, "A study on block assembly planning using simulation-based optimization in shipbuilding," *Journal of Ship Production and Design*, vol. 35, no. 1, pp. 60–71, 2019.